Lesson 3 – Arithmetic and User Input

ReDI School · Introduction to Programming

What did we learn last week?

- A **variable** stores a value: `name = "Sara"`, `age = 25`
- Four types: `str`, `int`, `float`, `bool`
- `type()` tells you the type · `len()` counts characters
- `str()`, `int()`, `float()` convert between types
- `input()` reads from the keyboard — always returns a string

Any questions before we start?

Today's Plan

Part	Topic
Part 1	Arithmetic operators
	Order of operations
	abs() and round()
☕	Break
Part 2	User input with input()
	Type conversion
☕	Break

Part 1 – Arithmetic

Arithmetic operators

Operator	What it does	Example	Result
`+`	Addition	`10 + 3`	`13`
`-`	Subtraction	`10 - 3`	`7`
`*`	Multiplication	`10 * 3`	`30`
`/`	Division	`10 / 3`	`3.333...`
`//`	Whole number division	`10 // 3`	`3`
`%`	Remainder	`10 % 3`	`1`
`**`	Power	`2 ** 8`	`256`

Arithmetic in action

```
# Shopping total
price    = 2.99
quantity = 4
total    = price * quantity
print(total)           # 11.96

# Is a number even? Remainder is 0 if yes
number = 8
print(number % 2)      # 0 — even!
number = 7
print(number % 2)      # 1 — odd!

# Update a variable with itself
score  = 10
score  = score + 5     # score is now 15
score += 3             # shortcut — score is now 18
print(score)           # 18
```

Real world — arithmetic is everywhere

Every time you buy something online, Python-style arithmetic runs in the background.

```
# Online shop checkout
item_price   = 29.99
quantity     = 3
discount_pct = 10        # 10% off
shipping     = 4.90

subtotal = item_price * quantity
discount = subtotal * (discount_pct / 100)
total    = round(subtotal - discount + shipping, 2)
print("Total: €" + str(total))    # Total: €85.87
```

✏️ Quick Exercise — Arithmetic

1. Create `a = 17` and `b = 5`. Print the result of: `a + b`, `a - b`, `a * b`, `a // b`, `a % b`
2. What is `2 ** 10`? Print it.
3. You have 30 sweets and 7 friends. How many does each friend get? How many are left over?

⏱️ 8 minutes

Quick Exercise — Solution

```
a = 17
b = 5

print(a + b)     # 22
print(a - b)     # 12
print(a * b)     # 85
print(a // b)    # 3
print(a % b)     # 2

print(2 ** 10)   # 1024

sweets  = 30
friends = 7
print(sweets // friends)   # 4 each
print(sweets % friends)    # 2 left over
```

Order of operations

- An **expression** is code that becomes one value, like `15 + 13`.
- Python **evaluates** (works out) the right side of `=` first, then stores the result in `a`.
- `+` is **left associative**: Python groups it from the left, so the first pair goes first.

Your code

```
a = 3
a = 15 + 13 + 17 + a + 18
```

How Python evaluates line 2

```
a = ((15 + 13) + 17 + a + 18)   # Start with the first pair
a = ((28 + 17) + a + 18)        # 15 + 13 = 28
a = ((45 + a) + 18)             # 28 + 17 = 45
a = ((45 + 3) + 18)             # a is still 3
a = (48 + 18)                   # 45 + 3 = 48
a = 66                          # 48 + 18 = 66, store it in a
```

Order of operations

Brackets run first, then powers, then multiply and divide, then add and subtract.

```
print(2 + 3 * 4)       # 14  — multiply first
print((2 + 3) * 4)     # 20  — brackets first
```

⚠️ Forgetting brackets is the most common mistake with averages:

```
score1 = 80
score2 = 90
score3 = 70

# ❌ Wrong — only score3 gets divided
average = score1 + score2 + score3 / 3
print(average)           # 193.3 — wrong!

# ✅ Correct — add everything first, then divide
average = (score1 + score2 + score3) / 3
print(average)           # 80.0
```

✏️ Quick Exercise — Fix the average

This code prints the wrong answer. Fix it.

```
score1 = 70
score2 = 85
score3 = 90

average = score1 + score2 + score3 / 3
print(average)
```

Bonus: use `round()` to print the average to 1 decimal place.

⏱️ 5 minutes

Quick Exercise – Solution

```
score1 = 70
score2 = 85
score3 = 90

# Fix: brackets around the sum
average = (score1 + score2 + score3) / 3
print(average)               # 81.666...

# Bonus: round to 1 decimal place
print(round(average, 1))     # 81.7
```

abs() and round()

```
# abs() — removes the minus sign, always positive
print(abs(-15))        # 15
print(abs(7))          # 7

temp = -8
print(abs(temp))       # 8  — "8 degrees below zero"
```

```
# round() — round to nearest whole number
print(round(3.7))          # 4
print(round(3.2))          # 3

# round(number, decimal places)
print(round(9.999, 2))     # 10.0
print(round(2.49, 1))      # 2.5

# Useful for prices
price = 49.99
final = round(price * 0.85, 2)
print(final)               # 42.49
```

Expression vs statement

Expression: code that becomes one value

```
15 + 13            # 28
abs(-8)            # 8
round(2.49, 1)     # 2.5
```

Statement: one complete instruction

```
a = 3              # store 3 in a
total = a + 5      # store 8 in total
print(total)       # show 8
```

Statements often contain expressions. In `total = a + 5`, Python evaluates the expression `a + 5` first. Then the statement stores the result in `total`.

☕ Break – 10 minutes

Back in 10 minutes

Part 2 – User Input

input()

`input()` pauses the program and waits for the user to type something:

```
name = input("What is your name? ")
print("Hello, " + name + "!")

city = input("Where are you from? ")
print(city + " sounds like a great place!")
```

```
What is your name? Sara
Hello, Sara!
Where are you from? Berlin
Berlin sounds like a great place!
```

The text inside `input("...")` is the prompt. Always add a space at the end so the cursor is not right against the text.

input() always returns a string

⚠️ Even if the user types a number, `input()` gives you a string. You must convert it before doing maths.

```
# ❌ This crashes
age = input("How old are you? ")
print(age + 1)
# TypeError: can only concatenate str (not "int") to str

# ✅ Convert to int first
age = int(input("How old are you? "))
print(age + 1)          # works!

# ✅ For decimal numbers use float
height = float(input("Your height in metres? "))
print(height * 100, "cm")
```

✏️ Quick Exercise — input()

1. Ask the user for their name. Print: `Hello, [name]! Nice to meet you.`
2. Ask the user for their age. Print: `In 10 years you will be [age + 10].`
3. Ask for a price. Print it doubled.

⏱️ 8 minutes

Quick Exercise – Solution

```
# 1. Greeting
name = input("What is your name? ")
print("Hello, " + name + "! Nice to meet you.")

# 2. Age in 10 years
age = int(input("How old are you? "))
print("In 10 years you will be", age + 10)

# 3. Double a price
price = float(input("Enter a price: "))
print("Doubled:", round(price * 2, 2))
```

Converting between types

```
# str → int
age = int("25")
print(age + 1)          # 26

# str → float
price = float("9.99")
print(price * 2)        # 19.98

# number → str  (needed when joining with +)
age = 25
print("I am " + str(age) + " years old")

# int() cuts the decimal — does NOT round
print(int(3.9))         # 3, not 4!
print(int(3.1))         # 3
```

Use `int()` or `float()` after `input()` when you need to do maths.
Use `str()` when joining a number into a sentence with `+`.

Common type errors

```
# ❌ Cannot add string and number
age = 25
print("I am " + age)           # TypeError!

# ✅ Convert to string first
print("I am " + str(age))      # I am 25
print("I am", age)             # I am 25 — comma works too
```

```
# ❌ input() gives a string — can't multiply by float
quantity = input("How many? ")
total = quantity * 2.99         # TypeError!

# ✅ Convert first
quantity = int(input("How many? "))
total = quantity * 2.99         # works!
```

✏️ Quick Exercise – VAT calculator

1. Ask the user to enter a price
2. Add 21% VAT: `total = price * 1.21`
3. Round to 2 decimal places and print: `Price with VAT: €[total]`

⏱️ 8 minutes

Quick Exercise – Solution

```
price = float(input("Enter a price (€): "))
total = round(price * 1.21, 2)
print("Price with VAT: €" + str(total))
```

⚠️ Common Mistakes

❌ **Using input() result as a number without converting**

`age = input("Age: ")` then `age + 1` crashes. Always wrap with `int()` or `float()`.

❌ **Forgetting brackets in averages**

`a + b + c / 3` only divides c. Write `(a + b + c) / 3`.

❌ **Expecting int() to round**

`int(3.9)` gives `3`, not `4`. Use `round()` when you want rounding.

❌ **Capitalised built-ins**

`Round()`, `Abs()`, `Int()` all crash. Python built-ins are always lowercase.

☕ Break – 10 minutes

Back in 10 minutes – mini project time!

Part 3 – Mini Project

🛠️ Mini Project – Personal Calculator

Ask the user for **two numbers**. Then print all of these:

1. Sum
2. Difference
3. Product
4. Quotient (rounded to 2 decimal places)
5. Remainder when the first is divided by the second
6. First number to the power of the second

⏱️ 20 minutes

What we learned today

- **7 arithmetic operators:** `+` `-` `*` `/` `//` `%` `**`
- **Brackets** come first in calculations — always use them for averages
- `abs()` removes the minus sign · `round()` controls decimal places
- `input()` always returns a string — convert with `int()` or `float()` before doing maths
- `str()` converts a number to text when joining with `+`

📚 Homework

1. **All operators** — create `a = 17`, `b = 5`. Print all 7 results with a label each.
2. **Average** — create 3 test scores. Calculate and print the average rounded to 1 decimal.
3. **abs() practice** — create two temperature variables. Print the difference using `abs()` so it is always positive.
4. **Discount calculator** — ask for a price and a discount percentage. Print the final price after the discount, rounded to 2 decimal places.
5. **BMI calculator** — ask for weight (kg) and height (m). Calculate `weight / height ** 2`, print rounded to 1 decimal.
6. **Seconds converter** — ask for a number of seconds. Print how many full minutes and leftover seconds that is. Use `//` and `%`.
7. **Personal calculator** — extend today's mini project to also print the integer division result.

📅 Next Week – Strings, Conditions and Logic

Next lesson we will look at:

- String methods: `.upper()`, `.lower()`, `.strip()`, `.replace()`
- Checking string content: `.isdigit()`, `.isalpha()`
- Slicing strings: `name[0:3]`
- f-strings: `f"Hello, {name}!"`
- `if` conditions: run code only when a condition is true
- Logic: combine conditions with `and`, `or`, and `not`

Great work today! 🎉

See you next week.